BERKAS JAWABAN UJIAN TENGAH SEMESTER (UTS)

Mata Kuliah: Pemrograman / Coding – Departemen Akuntansi FEB Universitas Diponegoro

Program Studi / Semester	S1 Reguler Akuntansi / Gasal TA 2026/2027
Dosen Pengampu	Totok Dewayanto
Nama	Raju Alvin Putra Batuara
NIM	12030124140290
Kelas	E
Link Repositori	https://github.com/rajubatuara-lang/uts-pengkodean-raju
Link Purwarupa Web	https://rajubatuara-lang.github.io/uts-pengkodean-raju/soal_3/

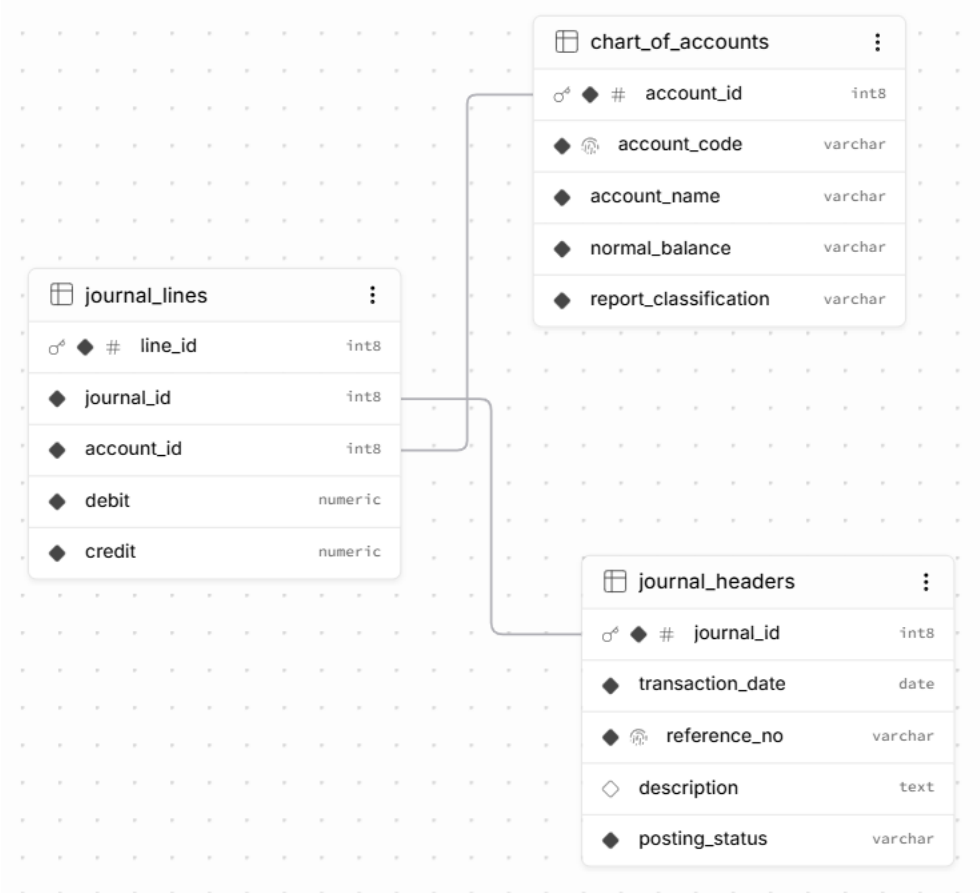
SOAL 1: Desain Arsitektur Data, Normalisasi, & Penegakan Asas Berpasangan [Bobot: 25%]

Analisis solusi terstruktur untuk pembangunan modul General Ledger mandiri guna mengatasi anomali saldo tidak seimbang (unbalanced balance sheet) dengan penegakan aturan double-entry pada tingkatan basis data PostgreSQL.

1.1 Perancangan Skema Relasional (ERD)

Untuk menjamin integritas transaksi berpasangan (double-entry bookkeeping), sistem menggunakan tiga tabel utama dengan spesifikasi kardinalitas sebagai berikut:

- **1. chart_of_accounts:** Menyimpan daftar akun standar akuntansi beserta sifat saldo normal dan klasifikasi laporan keuangan.
- **2. journal_headers:** Menyimpan metadata transaksi jurnal tingkat atas seperti tanggal transaksi, nomor bukti/referensi, deskripsi, dan status posting.
- **3. journal_lines:** Menyimpan rincian transaksi debit dan kredit secara atomik yang merujuk ke header transaksi dan akun spesifik.



Hubungan Relasi & Kardinalitas Entitas:

Relasi Entitas	Kardinalitas	Penjelasan & Aturan Bisnis

journal_headers (1) ——— (N) journal_lines One-to-Many	Satu header transaksi wajib memiliki minimal 2 baris rincian (lines) untuk memenuhi asas keseimbangan Debit = Kredit.
chart_of_accounts (1) ——— (N) journal_lines One-to-Many	Satu akun dapat dirujuk oleh banyak baris transaksi di berbagai jurnal tanpa duplikasi master data.

1.2 Kekangan Integritas (Database Integrity & Constraints DDL PostgreSQL)

Berikut adalah skrip DDL PostgreSQL lengkap yang menerapkan Primary Key, Foreign Key (ON DELETE RESTRICT), serta CHECK constraint komprehensif:

```
-- 1. Tabel Chart of Accounts (Daftar Akun Master)
CREATE TABLE chart_of_accounts (
   CREATE TABLE chart_of_accounts (
   account_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   account_code VARCHAR(20) NOT NULL UNIQUE,
   account_name VARCHAR(100) NOT NULL,
   normal_balance VARCHAR(10) NOT NULL
      CHECK (normal_balance IN ('DEBIT', 'CREDIT')),
   report_classification VARCHAR(30) NOT NULL
);

-- 2. Tabel Journal Headers (Header Transaksi Jurnal)
CREATE TABLE journal_headers (
   journal_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   transaction_date DATE NOT NULL,
   reference_no VARCHAR(50) NOT NULL UNIQUE,
   description TEXT,
   posting_status VARCHAR(10) NOT NULL DEFAULT 'DRAFT'
      CHECK (posting_status IN ('DRAFT', 'POSTED'))
);

-- 3. Tabel Journal Lines (Rincian Debit-Kredit Jurnal)
CREATE TABLE journal_lines (
   line_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

   journal_id BIGINT NOT NULL
      REFERENCES journal_headers(journal_id)
      ON DELETE RESTRICT,
```

```
  account_id BIGINT NOT NULL
    REFERENCES chart_of_accounts(account_id)
    ON DELETE RESTRICT,

  debit NUMERIC(15, 2) NOT NULL DEFAULT 0
    CHECK (debit >= 0),

  credit NUMERIC(15, 2) NOT NULL DEFAULT 0
    CHECK (credit >= 0),

  CHECK (debit > 0 OR credit > 0)
);
```

1.3 Analisis Audit & Keamanan Finansial (Audit Trail & 3NF)

1. **Keandalan Jejak Audit (Audit Trail):** Pemisahan Header dan Line memisahkan konteks transaksi (kapan, nomor referensi, siapa pencipta, status posting) dari angka mutasi keuangan. Penguncian status posting (POSTED) dan hashing SHA-256 dapat diterapkan terpusat pada tingkat Header, sehingga percobaan manipulasi pada level Line akan terdeteksi seketika.

2. **Kepatuhan Bentuk Normal Ketiga (3NF):** Dalam model tabel tunggal (flat journal table), data transaksi seperti tanggal dan nomor bukti diulang pada setiap baris debit dan kredit, menimbulkan redundansi. Pemisahan ke dalam Header-Line menghilangkan ketergantungan parsial dan transitif. Seluruh atribut Header bergantung penuh hanya pada header_id, dan atribut Line bergantung pada line_id, memenuhi syarat 3NF serta Mencegah Anomali Pembaruan (Update, Insert, Delete Anomalies).

ANALISIS SOAL 1

Desain Arsitektur Data, Normalisasi, dan Penegakan Asas Berpasangan

1. Tujuan Perancangan Database

Perancangan database General Ledger bertujuan untuk membangun sistem pencatatan jurnal akuntansi yang terstruktur, konsisten, dan dapat dipertanggungjawabkan. Sistem harus mampu menjaga integritas data serta mengurangi risiko terjadinya jurnal yang tidak valid akibat hanya mengandalkan validasi pada antarmuka pengguna (*frontend validation*).

Untuk mengatasi masalah tersebut, validasi diterapkan pada tingkat database melalui penggunaan *Primary Key*, *Foreign Key*, dan *CHECK constraint*. Dengan demikian, database memiliki aturan integritas yang dapat diterapkan meskipun data dimasukkan melalui jalur selain antarmuka aplikasi.

2. Analisis Struktur Tabel

Database dirancang menggunakan tiga tabel utama, yaitu chart_of_accounts, journal_headers, dan journal_lines.

2.1 Tabel chart_of_accounts

Tabel chart_of_accounts digunakan untuk menyimpan daftar akun yang digunakan dalam pencatatan transaksi perusahaan.
Kolom-kolom utamanya meliputi:

- account_id: identitas unik setiap akun dan berfungsi sebagai *Primary Key*.
- account_code: kode akun yang digunakan untuk membedakan setiap akun.
- account_name: nama akun, misalnya Kas, Piutang Usaha, dan Beban Listrik.
- normal_balance: menunjukkan saldo normal akun, yaitu debit atau kredit.
- report_classification: menunjukkan klasifikasi akun dalam laporan keuangan.

Tabel ini menjadi referensi bagi pencatatan jurnal sehingga setiap baris jurnal dapat dihubungkan dengan akun yang valid.

2.2 Tabel journal_headers

Tabel journal_headers digunakan untuk menyimpan informasi umum dari suatu transaksi jurnal.
Kolom-kolom utamanya meliputi:

- journal_id: identitas unik jurnal dan berfungsi sebagai *Primary Key*.
- transaction_date: tanggal transaksi.
- reference_no: nomor referensi atau bukti transaksi.
- description: keterangan mengenai transaksi.
- posting_status: status jurnal, misalnya DRAFT atau POSTED.

Pemisahan informasi umum ke dalam tabel header memungkinkan satu transaksi memiliki beberapa baris jurnal tanpa harus mengulang informasi tanggal, nomor referensi, dan deskripsi pada setiap baris.

2.3 Tabel journal_lines

Tabel journal_lines digunakan untuk menyimpan rincian akun dan nominal debit maupun kredit dari suatu jurnal.
Kolom-kolom utamanya meliputi:

- line_id: identitas unik setiap baris jurnal dan berfungsi sebagai *Primary Key*.
- journal_id: *Foreign Key* yang menghubungkan baris jurnal dengan tabel journal_headers.
- account_id: *Foreign Key* yang menghubungkan baris jurnal dengan tabel chart_of_accounts.
- debit: nominal debit pada baris jurnal.
- credit: nominal kredit pada baris jurnal.

Tabel ini memungkinkan satu jurnal memiliki beberapa baris pencatatan yang mewakili akun-akun yang terlibat dalam transaksi.

3. Analisis Hubungan Antarentitas (ERD)

Hubungan antarentitas dalam database dirancang sebagai berikut:

1. Satu data journal_headers dapat memiliki banyak data journal_lines, sehingga hubungan keduanya adalah satu-ke-banyak (*one-to-many*).
2. Satu data chart_of_accounts dapat digunakan pada banyak data journal_lines, sehingga hubungan keduanya juga satu-ke-banyak (*one-to-many*).

Secara sederhana, hubungan tersebut dapat digambarkan sebagai berikut:

chart_of_accounts
|
| 1
|
| N
journal_lines
|
| N
|
| 1
journal_headers

Pada implementasi database, journal_lines.journal_id mengacu pada journal_headers.journal_id, sedangkan journal_lines.account_id mengacu pada chart_of_accounts.account_id.

Hubungan tersebut memastikan bahwa setiap baris jurnal merujuk pada jurnal induk dan akun yang terdaftar di database.

4. Analisis Integritas Database dan Constraints

Integritas database diperlukan untuk memastikan data yang disimpan memenuhi aturan struktur dan pencatatan yang telah ditentukan.

4.1 Primary Key

Primary Key digunakan untuk mengidentifikasi setiap baris data secara unik. Pada rancangan ini, setiap tabel memiliki *Primary Key* masing-masing, yaitu account_id, journal_id, dan line_id.

Penggunaan *Primary Key* mencegah identitas data menjadi ambigu dan memudahkan hubungan antarentitas.

4.2 Foreign Key

Foreign Key digunakan untuk menjaga hubungan antara tabel yang saling berkaitan.

Pada tabel journal_lines, kolom journal_id mengacu pada tabel journal_headers, sedangkan account_id mengacu pada tabel chart_of_accounts.

Dengan demikian, baris jurnal tidak dapat merujuk pada jurnal induk atau akun yang tidak tersedia dalam tabel referensi.

4.3 ON DELETE RESTRICT

Klausa ON DELETE RESTRICT digunakan untuk mencegah penghapusan data induk yang masih digunakan oleh data lain.

Sebagai contoh, akun yang masih direferensikan oleh baris jurnal tidak dapat dihapus secara langsung. Aturan ini membantu menjaga konsistensi hubungan data dan mengurangi risiko hilangnya referensi transaksi.

4.4 CHECK Constraint

Pada tabel journal_lines, digunakan beberapa aturan validasi nominal:

- CHECK (debit >= 0) memastikan nominal debit tidak negatif.
- CHECK (credit >= 0) memastikan nominal kredit tidak negatif.
- CHECK (debit > 0 OR credit > 0) memastikan debit dan kredit tidak sama-sama bernilai nol pada satu baris jurnal.

Ketiga aturan tersebut merupakan validasi pada tingkat baris. Namun, aturan tersebut belum menjamin bahwa jumlah seluruh debit sama dengan jumlah seluruh kredit dalam satu jurnal.

Oleh karena itu, keseimbangan jurnal perlu diperiksa pada tingkat transaksi jurnal secara keseluruhan, terutama sebelum jurnal diubah menjadi berstatus POSTED. Pemeriksaan tersebut dapat diterapkan melalui fungsi atau trigger database yang dirancang untuk memvalidasi total debit dan kredit.

5. Analisis Normalisasi Bentuk Normal Ketiga (3NF)

Normalisasi merupakan proses pengorganisasian data untuk mengurangi pengulangan dan mencegah anomali dalam penyimpanan maupun perubahan data.

Pemisahan tabel chart_of_accounts, journal_headers, dan journal_lines mendukung rancangan database yang lebih terstruktur.

Beberapa manfaat pemisahan tersebut adalah:

1. Informasi akun disimpan pada tabel chart_of_accounts, sehingga nama dan kode akun tidak perlu didefinisikan ulang pada setiap baris jurnal.
2. Informasi umum transaksi disimpan pada journal_headers, sehingga tanggal, nomor referensi, dan deskripsi tidak perlu diulang pada setiap baris jurnal.
3. Informasi nominal debit dan kredit disimpan pada journal_lines, sehingga rincian transaksi dapat terdiri dari beberapa akun.
4. Perubahan informasi referensi dapat dikelola secara terpusat pada tabel yang sesuai.

Bentuk normal ketiga (3NF) mensyaratkan bahwa tabel telah memenuhi bentuk normal kedua dan tidak memiliki ketergantungan transitif atribut nonkunci terhadap *Primary Key*. Dengan kata lain, atribut nonkunci harus bergantung pada kunci, keseluruhan kunci, dan tidak bergantung secara transitif pada atribut nonkunci lainnya.

Pemisahan tabel pada rancangan ini mendukung tujuan 3NF karena setiap tabel memiliki fungsi dan kelompok atribut yang berbeda. Namun, kepatuhan formal terhadap 3NF tetap harus dinilai berdasarkan ketergantungan fungsional atribut pada setiap tabel, bukan hanya berdasarkan jumlah tabel yang digunakan.

6. Analisis Audit Trail dan Keamanan Finansial

Audit trail merupakan jejak yang memungkinkan transaksi dan aktivitas terkait ditelusuri untuk keperluan pemeriksaan. Pemisahan header dan detail jurnal membantu proses audit karena auditor dapat mengidentifikasi informasi umum suatu transaksi melalui journal_headers dan menelusuri seluruh akun beserta nominalnya melalui journal_lines. Sebagai contoh, ketika auditor memeriksa transaksi pembayaran listrik, auditor dapat menelusuri nomor bukti, tanggal, dan deskripsi transaksi, kemudian memeriksa akun yang didebit serta akun yang dikredit. Meskipun demikian, pemisahan tabel saja belum cukup untuk membentuk audit trail yang lengkap. Sistem sebaiknya dilengkapi dengan pencatatan identitas pengguna, waktu pembuatan, riwayat perubahan, status posting, serta pembatasan akses terhadap transaksi yang telah diposting. Penerapan kontrol tersebut membantu meningkatkan keterlacakan transaksi dan mengurangi risiko perubahan data tanpa otorisasi.

7. Contoh Penerapan Asas Berpasangan (Double-Entry)

Sebagai contoh, perusahaan melakukan pembayaran biaya listrik secara tunai sebesar Rp1.000.000.

Pencatatan jurnalnya adalah sebagai berikut:

Akun	Debit	Kredit
Beban Listrik	Rp1.000.000	Rp0
Kas	Rp0	Rp1.000.000
Total	**Rp1.000.000**	**Rp1.000.000**

Transaksi tersebut dicatat dalam satu baris header dan dua baris detail jurnal. Prinsip pencatatan berpasangan mengharuskan total debit sama dengan total kredit untuk setiap jurnal yang lengkap dan seimbang. Dalam contoh ini, total debit sebesar Rp1.000.000 sama dengan total kredit sebesar Rp1.000.000. Database harus memastikan bahwa baris jurnal menggunakan akun yang valid dan nominalnya memenuhi constraint. Selain itu, proses posting harus memvalidasi keseimbangan seluruh baris jurnal sebelum transaksi disahkan.

8. Kesimpulan

Rancangan database General Ledger menggunakan tiga tabel utama, yaitu chart_of_accounts, journal_headers, dan journal_lines, untuk memisahkan data referensi akun, informasi umum transaksi, dan rincian jurnal. Penerapan *Primary Key*, *Foreign Key*, ON DELETE RESTRICT, dan CHECK constraint membantu menjaga integritas struktur serta nominal data. Pemisahan header dan detail juga mendukung normalisasi dan memudahkan penelusuran transaksi. Namun, integritas nominal pada setiap baris tidak secara otomatis menjamin keseimbangan total debit dan kredit. Oleh karena itu, sistem perlu memiliki validasi tambahan sebelum posting, serta mekanisme audit trail dan pembatasan akses agar pencatatan akuntansi menjadi lebih konsisten, terlacak, dan dapat dipertanggungjawabkan.

SOAL 2: Otomasi Bisnis, Stored Procedures, & Database Trigger Persediaan [Bobot: 40%]

Implementasi fungsi otomasi bisnis untuk pengelolaan persediaan sistem perpetual di PT Diponegoro Logistik Utama untuk mencegah selisih stok opname dan menangani akses bersamaan (concurrency).

2.1 Otomatisasi Mutasi via Database Trigger

Berikut adalah kode lengkap PostgreSQL Trigger Function dan Trigger yang secara otomatis mengupdate stok dan membatalkan transaksi jika persediaan tidak mencukupi:

-- 1. Fungsi Trigger Utama

```
CREATE OR REPLACE FUNCTION trg_update_stock_and_journal()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_stock INT;
BEGIN
    IF NEW.qty = 0 THEN
        RAISE EXCEPTION 'Kuantitas tidak boleh nol';
    END IF;

    SELECT stock_qty
    INTO v_stock
    FROM inventory_items
    WHERE item_id = NEW.item_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Barang tidak ditemukan';
    END IF;

    IF NEW.trx_type = 'PURCHASE' THEN

        IF NEW.qty < 0 THEN
            RAISE EXCEPTION
                'Kuantitas pembelian harus positif';
        END IF;

        UPDATE inventory_items
        SET stock_qty = stock_qty + NEW.qty
        WHERE item_id = NEW.item_id;
```

```
    ELSIF NEW.trx_type = 'SALE' THEN

        IF NEW.qty < 0 THEN
            RAISE EXCEPTION
                'Kuantitas penjualan harus positif';
        END IF;

        IF v_stock < NEW.qty THEN
            RAISE EXCEPTION
                'Stok tidak mencukupi';
        END IF;

        UPDATE inventory_items
        SET stock_qty = stock_qty - NEW.qty
        WHERE item_id = NEW.item_id;

    ELSIF NEW.trx_type = 'ADJUSTMENT' THEN

        UPDATE inventory_items
        SET stock_qty = stock_qty + NEW.qty
        WHERE item_id = NEW.item_id
          AND stock_qty + NEW.qty >= 0;

        IF NOT FOUND THEN
            RAISE EXCEPTION
                'Penyesuaian menyebabkan stok negatif';
        END IF;

    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER inventory_stock_trigger
BEFORE INSERT ON inventory_transactions
FOR EACH ROW
EXECUTE FUNCTION trg_update_stock_and_journal();

INSERT INTO inventory_transactions
    (item_id, trx_type, qty)
VALUES
    ('BRG001', 'PURCHASE', 5);
```

```
    SELECT item_id, item_name, stock_qty
FROM inventory_items
WHERE item_id = 'BRG001';

INSERT INTO inventory_transactions
    (item_id, trx_type, qty)
VALUES
    ('BRG001', 'SALE', 3);

    INSERT INTO inventory_transactions
    (item_id, trx_type, qty)
VALUES
    ('BRG001', 'SALE', 100);
```

2.2 Penanganan Concurrency (Race Condition / Overselling)

Anomali Race Condition timbul ketika dua transaksi penjualan eksekusi secara bersamaan pada item dengan stok 1 unit. Tanpa penguncian baris, kedua sesi membaca stock_qty = 1 dan keduanya berhasil melakukan pengurangan stok, mengakibatkan overselling (stok -1).
Solusi teknis yang digunakan adalah Row-Level Locking melalui perintah SELECT ... FOR UPDATE di dalam fungsi trigger dengan level isolasi READ COMMITTED / SERIALIZABLE. Sesi kedua akan dipaksa menunggu (wait) hingga sesi pertama melakukan COMMIT/ROLLBACK.

```
-- Skenario Eksekusi Kasir A (Berhasil):
BEGIN ISOLATION LEVEL READ COMMITTED;

-- Penguncian baris barang eksklusif
SELECT item_id, stock_qty FROM inventory_items WHERE item_id = 'ITM-001' FOR UPDATE;

INSERT INTO inventory_transactions (item_id, trx_type, qty)
VALUES ('ITM-001', 'SALE', 1);

COMMIT; -- Stok terupdate menjadi 0

----------------------------------------------------------------------

-- Skenario Eksekusi Kasir B (Dijalankan Serentak):
BEGIN ISOLATION LEVEL READ COMMITTED;
```

```
-- Kasir B tertahan (WAIT) pada baris ini hingga Kasir A selesai COMMIT
SELECT item_id, stock_qty FROM inventory_items WHERE item_id = 'ITM-001' FOR UPDATE;

-- Setelah Kasir A COMMIT, Kasir B membaca stock_qty = 0
-- Perintah INSERT memicu trigger -> RAISE EXCEPTION 'Stok tidak mencukupi'
INSERT INTO inventory_transactions (item_id, trx_type, qty)
VALUES ('ITM-001', 'SALE', 1);

ROLLBACK; -- Transaksi Kasir B otomatis dibatalkan
```

SOAL 3: Integrasi Purwarupa Web Full-Stack & Orkestrasi AI Tools [Bobot: 35%]

Pengembangan purwarupa modul Voucher Entri Transaksi Kas Keluar berbasis Python Streamlit dan backend FastAPI terintegrasi aturan zero-discrepancy serta immutability.

3.1 Purwarupa UI & Verifikasi PSAK

a. Prompt AI Terpandu (Dokumentasi Prompting)

Prompt AI yang digunakan untuk mengonstruksi UI Streamlit:

Buat purwarupa website Sistem Voucher Entri Transaksi Kas Keluar untuk perusahaan. Gunakan HTML, CSS, dan JavaScript dengan tampilan profesional, bersih, responsif, dan cocok untuk sistem akuntansi.

Fitur yang wajib tersedia:

1. Form nomor voucher, tanggal transaksi, dan keterangan.
2. Tabel jurnal dinamis dengan kolom akun, debit, dan kredit.
3. Tombol tambah baris dan hapus baris jurnal.
4. Kalkulasi total debit dan total kredit secara otomatis ketika pengguna mengubah nominal.
5. Indikator status Balanced jika total debit sama dengan total kredit, dan Unbalanced jika berbeda.
6. Tombol Submit hanya aktif jika data wajib terisi, nominal valid, terdapat baris jurnal, dan total debit sama dengan total kredit.
7. Pisahkan kode menjadi index.html, style.css, dan app.js.
8. Tambahkan contoh jurnal pembayaran kas keluar.
9. Berikan validasi yang jelas dan mudah dipahami pengguna.

Jangan mengklaim bahwa aplikasi sudah terhubung ke database jika integrasi backend belum dibuat. Tampilkan kode lengkap untuk ketiga file tersebut.

b. Kode Purwarupa Aplikasi Web

> https://github.com/rajubatuara-lang/uts-pengkodean-raju/blob/main/soal_3/index.html

c. Ringkasan Verifikasi Aturan Bisnis PSAK (Pengakuan Kas & Setara Kas)

1. PSAK 2 (Laporan Arus Kas): Setiap pengeluaran kas wajib diakui bersamaan dengan pengakuan beban atau pelunasan kewajiban secara simetris (double-entry principle). Pengendalian real-time pada UI memutakhirkan posisi arus kas secara tepat waktu.

2. Pengendalian Interne & Auditability: Sistem melarang pengurangan kas sepihak tanpa adanya akun tandingan yang terverifikasi, menjamin bahwa arus kas keluar memiliki otorisasi dan keterlacakan penuh (auditability).

3.2 Mekanisme Jejak Audit & Immutability

a. Backend Function (FastAPI - Jurnal Pembalik & SHA-256 Checksum)

```
-- ============================================================
-- Schema: Voucher Kas Keluar
-- Jalankan script ini di Supabase SQL Editor
-- ============================================================

-- Tabel header voucher
CREATE TABLE IF NOT EXISTS voucher_headers (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    voucher_no  VARCHAR(50)  NOT NULL UNIQUE,
    trans_date  DATE         NOT NULL,
    description TEXT         NOT NULL,
    total_amount NUMERIC(15,2) NOT NULL DEFAULT 0,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- Tabel baris jurnal (detail)
CREATE TABLE IF NOT EXISTS voucher_lines (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    voucher_id  BIGINT       NOT NULL
                    REFERENCES voucher_headers(id)
                    ON DELETE CASCADE,
    account_id  VARCHAR(20)  NOT NULL,
    account_name VARCHAR(100) NOT NULL,
    debit       NUMERIC(15,2) NOT NULL DEFAULT 0 CHECK (debit >= 0),
    credit      NUMERIC(15,2) NOT NULL DEFAULT 0 CHECK (credit >= 0),
    CHECK (debit > 0 OR credit > 0)
);

-- Aktifkan Row Level Security
ALTER TABLE voucher_headers ENABLE ROW LEVEL SECURITY;
ALTER TABLE voucher_lines   ENABLE ROW LEVEL SECURITY;

-- Policy: izinkan semua operasi dari anon key (untuk purwarupa)
CREATE POLICY "Allow all for anon" ON voucher_headers
    FOR ALL TO anon USING (true) WITH CHECK (true);

CREATE POLICY "Allow all for anon" ON voucher_lines
    FOR ALL TO anon USING (true) WITH CHECK (true);
```

b. Database Immutability Trigger (PostgreSQL)

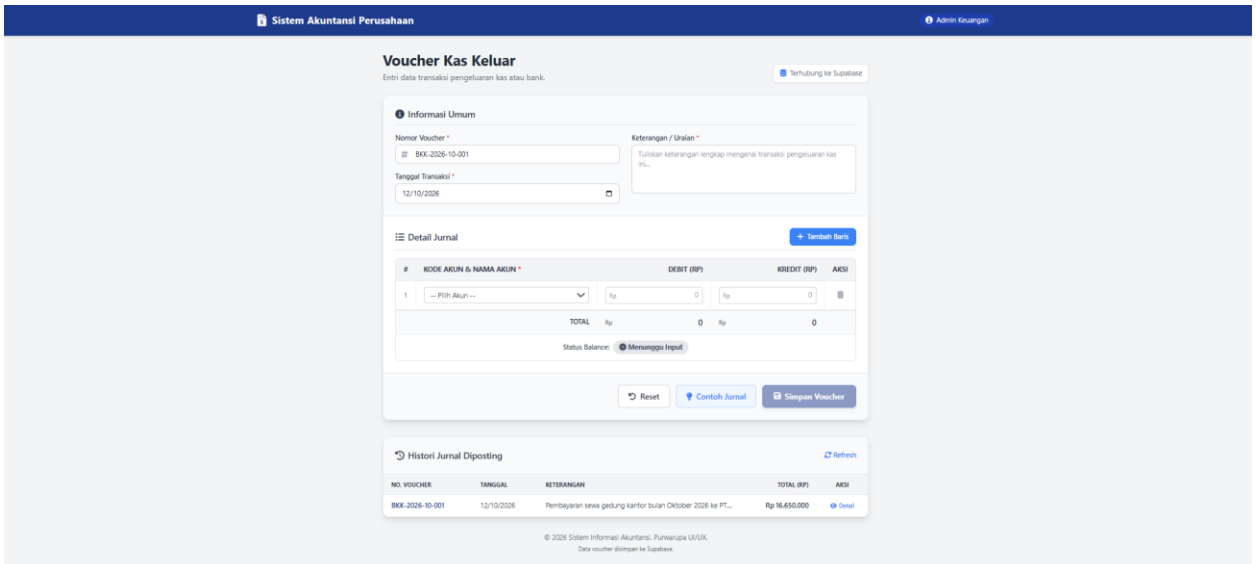
```
-- Trigger Function Mencegah Pengubahan / Penghapusan Data POSTED
CREATE OR REPLACE FUNCTION fn_prevent_posted_modification()
RETURNS TRIGGER AS $$
BEGIN
   IF OLD.posting_status = 'POSTED' THEN
      RAISE EXCEPTION 'Akses Ditolak: Transaksi berstatus POSTED tidak boleh diubah (UPDATE) atau dihapus (DELETE)! Gunakan Jurnal Pembalik (Reversing Entry).';
   END IF;
   RETURN OLD;
END;
$$ LANGUAGE plpgsql;

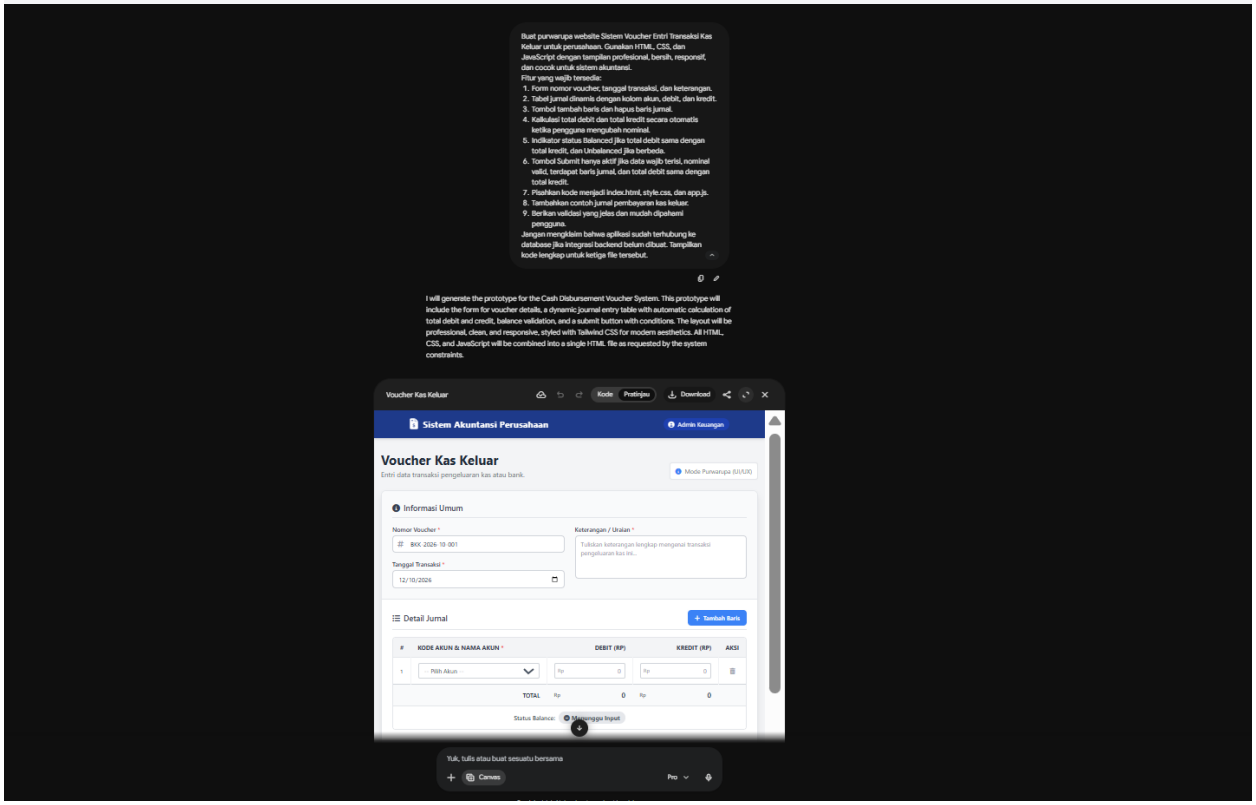
CREATE TRIGGER trg_protect_posted_headers
BEFORE UPDATE OR DELETE ON journal_headers
FOR EACH ROW
EXECUTE FUNCTION fn_prevent_posted_modification();
```

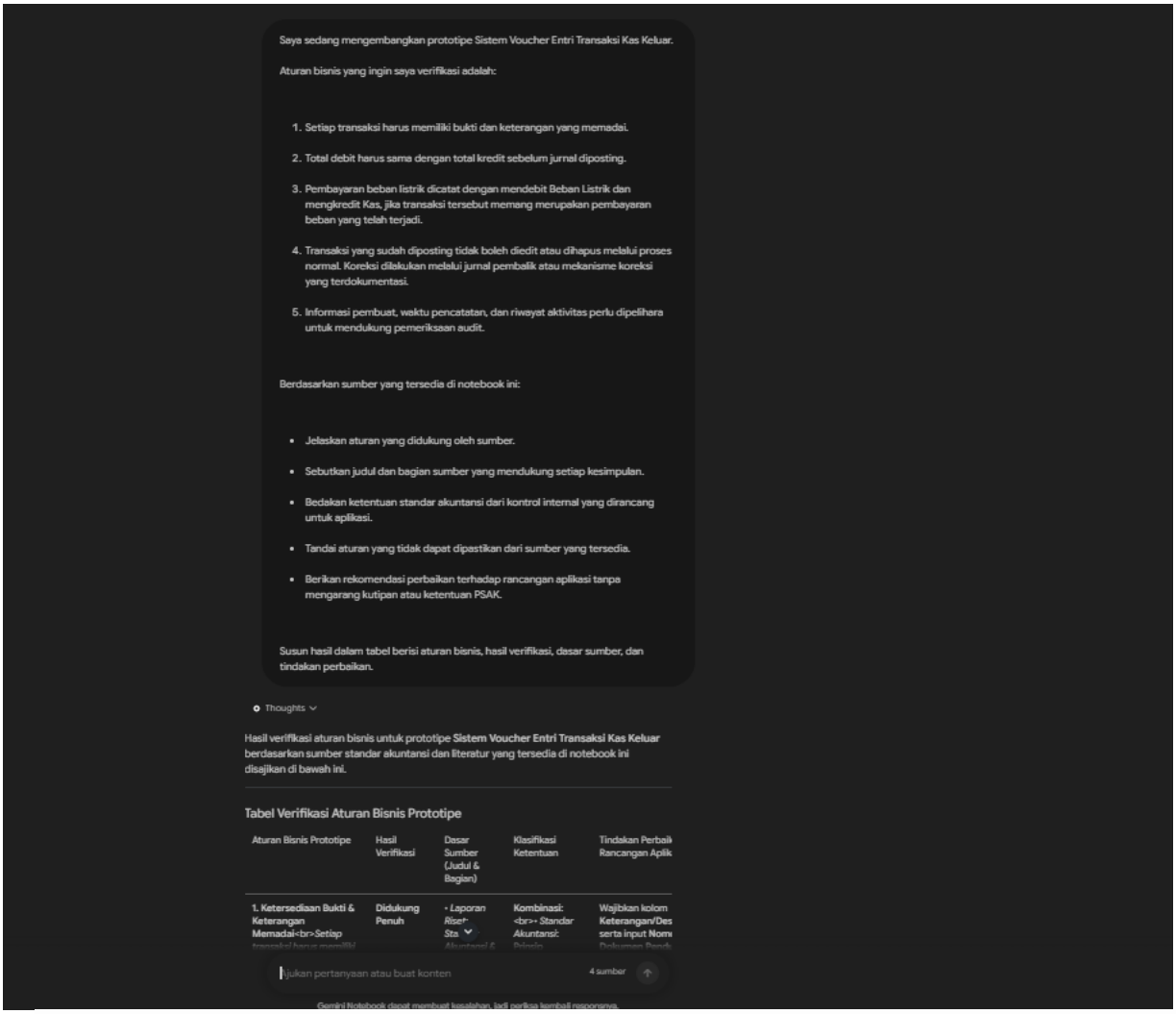
c. Parameter Integritas Data Finansial

- **created_by:** Menandai identitas pengguna/sistem yang menginput transaksi untuk penegakan akuntabilitas personal.
- **created_at:** Stempel waktu terpusat dari server (server timestamp) untuk menjamin urutan kronologis kejadian.
- **hash_checksum:** Nilai enkripsi SHA-256 yang memadukan parameter utama transaksi. Jika terjadi manipulasi langsung pada basis data, hash tidak akan cocok sehingga kecurangan terdeteksi seketika.

BEBERAPA TANGKAPAN LAYAR







Saya sedang mengembangkan prototipe Sistem Voucher Entri Transaksi Kas Keluar.

Aturan bisnis yang ingin saya verifikasi adalah:

1. Setiap transaksi harus memiliki bukti dan keterangan yang memadai.
2. Total debit harus sama dengan total kredit sebelum jurnal diposting.
3. Pembayaran beban listrik dicatat dengan mendebit Beban Listrik dan mengkredit Kas, jika transaksi tersebut memang merupakan pembayaran beban yang telah terjadi.
4. Transaksi yang sudah diposting tidak boleh diedit atau dihapus melalui proses normal. Koreksi dilakukan melalui jurnal pembalik atau mekanisme koreksi yang terdokumentasi.
5. Informasi pembuat, waktu pencatatan, dan riwayat aktivitas perlu dipelihara untuk mendukung pemeriksaan audit.

Berdasarkan sumber yang tersedia di notebook ini:

- Jelaskan aturan yang didukung oleh sumber.
- Sebutkan judul dan bagian sumber yang mendukung setiap kesimpulan.
- Bedakan ketentuan standar akuntansi dari kontrol internal yang dirancang untuk aplikasi.
- Tandai aturan yang tidak dapat dipastikan dari sumber yang tersedia.
- Berikan rekomendasi perbaikan terhadap rancangan aplikasi tanpa mengarang kutipan atau ketentuan PSAK.

Susun hasil dalam tabel berisi aturan bisnis, hasil verifikasi, dasar sumber, dan tindakan perbaikan.

Thoughts

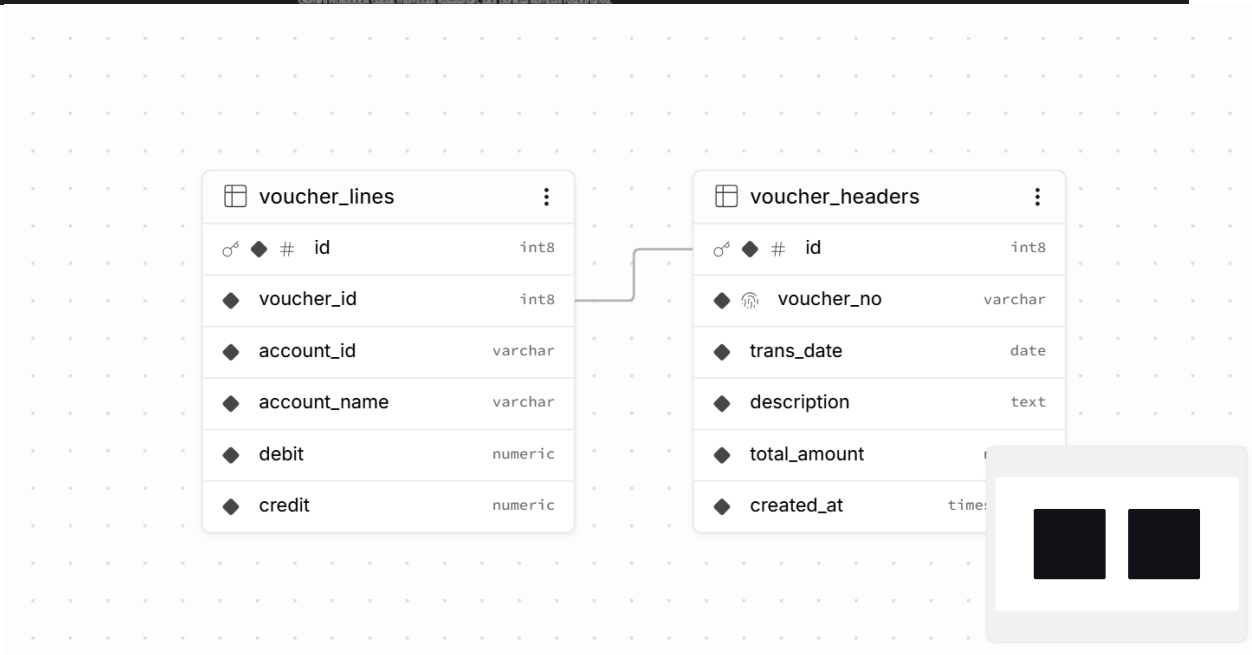
Hasil verifikasi aturan bisnis untuk prototipe **Sistem Voucher Entri Transaksi Kas Keluar** berdasarkan sumber standar akuntansi dan literatur yang tersedia di notebook ini disajikan di bawah ini.

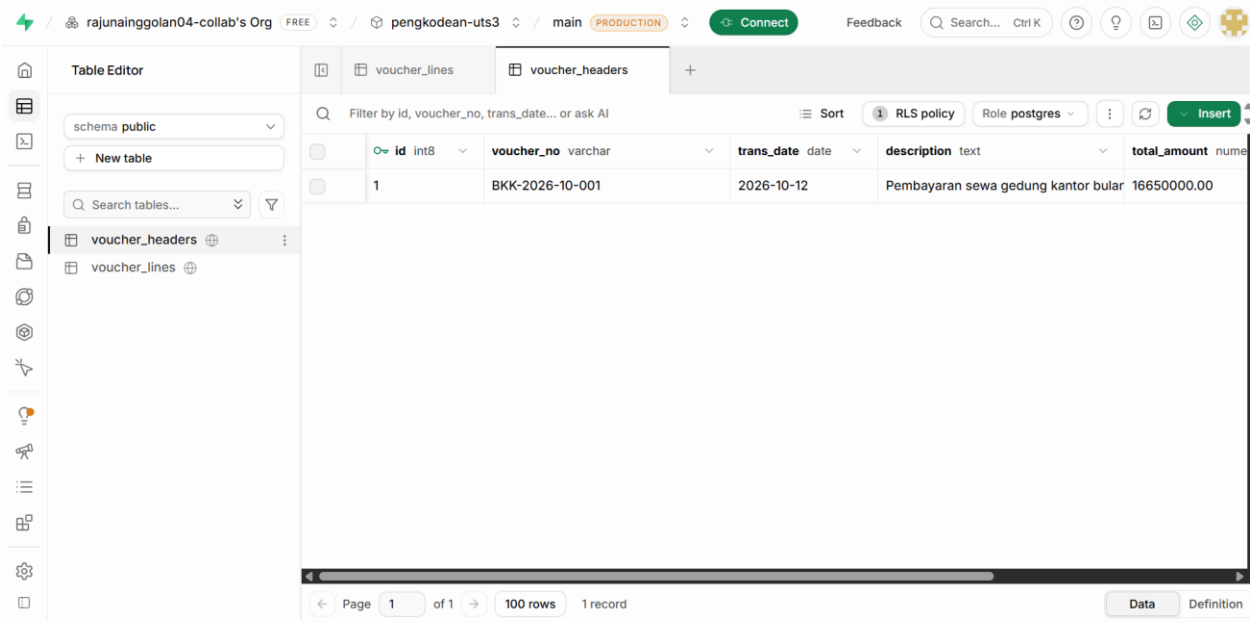
Tabel Verifikasi Aturan Bisnis Prototipe

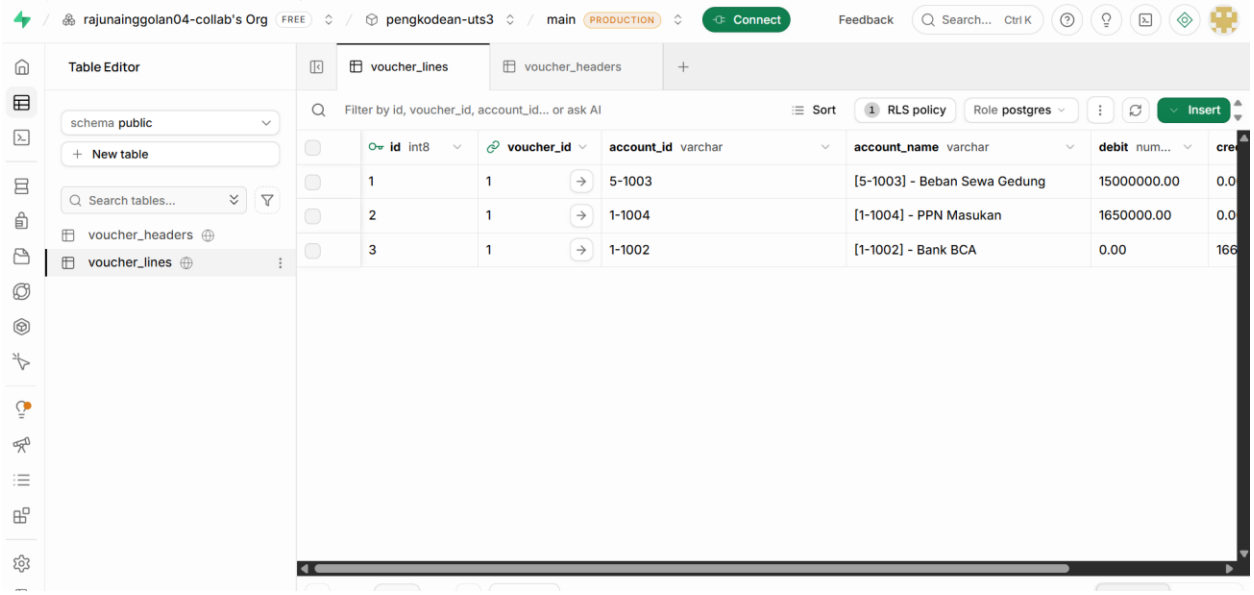
Aturan Bisnis Prototipe	Hasil Verifikasi	Dasar Sumber (Judul & Bagian)	Klasifikasi Ketentuan	Tindakan Perbaik Rancangan Aplik
1. Ketersediaan Bukti & Keterangan Memadai Setiap	Didukung Penuh	• Laporan Riset Sta	Kombinasi: • Standar Akuntansi:	Wajibkan kolom Keterangan/Des serta input Nom

jukan pertanyaan atau buat konten 4 sumber

Gemini Notebook dapat membuat kesalahan, jadi periksa kembali responsnya.




rajunainggolan04-collab's Org
FREE
pengkodean-uts3
main
PRODUCTION
Connect
Feedback
Search... Ctrl K
Table Editor
schema public
New table
Search tables...
voucher_headers
voucher_lines
voucher_lines
voucher_headers
Filter by id, voucher_no, trans_date... or ask AI
Sort
RLS policy
Role postgres
Insert
id int8
voucher_no varchar
trans_date date
description text
total_amount
1
BKK-2026-10-001
2026-10-12
Pembayaran sewa gedung kantor bular
16650000.00
Page 1 of 1
100 rows
1 record
Data
Definition


rajunainggolan04-collab's Org
FREE
pengkodean-uts3
main
PRODUCTION
Connect
Feedback
Search... Ctrl K
Table Editor
schema public
New table
Search tables...
voucher_headers
voucher_lines
voucher_lines
voucher_headers
Filter by id, voucher_id, account_id... or ask AI
Sort
RLS policy
Role postgres
Insert
id int8
voucher_id
account_id varchar
account_name varchar
debit num...
1 1 5-1003 [5-1003] - Beban Sewa Gedung 15000000.00
2 1 1-1004 [1-1004] - PPN Masukan 1650000.00
3 1 1-1002 [1-1002] - Bank BCA 0.00